FORMATION

# Analyse de données avec Python

Jour 2, après-midi
Visualisation avec matplotlib, seaborn et plotly

Cours et TP4 en alternance, sur les indicateurs consolidés du TP3

# Le déroulé de l'après-midi

1. **Données** — puis TP4, partie 0
2. **matplotlib** — puis TP4, partie 1
3. **seaborn** — puis TP4, partie 2

**Pause**

4. **plotly** — puis TP4, partie 3
5. **Bonnes pratiques** — puis TP4, partie 4
6. **Tableau de bord** — puis TP4, partie 5

**13h30**
début

**15h00**
pause

**17h00**
fin

Objectif : un tableau de bord de graphiques clairs, qui portent chacun un message.

# 01

# Les données et l'environnement

Deux tables | Trois bibliothèques | Deux agrégats

# 1 Les données et l'environnement

| Fichier | Sert à |
|---|---|
| `indicateurs_mois_region.parquet` | évolutions et comparaisons |
| `ventes_consolidees.parquet` | distributions |

```
from pathlib import Path

import matplotlib.pyplot as plt
import pandas as pd
import plotly.express as px
import seaborn as sns
```

Grâce au Parquet, les dates sont déjà des dates : aucune conversion à refaire.

Trois bibliothèques à ajouter

```
python -m pip install matplotlib seaborn plotly
python -m pip freeze > requirements.txt
```

Deux agrégats qui reviennent tout l'après-midi

```
mensuel = indicateurs.groupby("mois")["ca"].sum()
par_region = (indicateurs.groupby("region")["ca"]
              .sum().sort_values())
```

`mensuel` : le chiffre d'affaires de chaque mois. `par_region` : celui de chaque région, trié par ordre croissant.

# 1 Pratique : TP4, partie 0

Dans exercices/tp04-tableau-de-bord/tp04.ipynb, à compléter au fil de l'après-midi

Activer l'environnement, installer les trois bibliothèques, mettre à jour `requirements.txt`
Lancer `jupyter lab` dans le dossier du TP, exécuter la cellule de chargement

1. Question 0.1 : dimensions, premier et dernier mois, liste des régions

Question 0.2 : calculer `mensuel` et `par_region`

Résultat attendu de la question 0.2 : par_region

```
region
Bretagne                 41079.71
Occitanie                50318.18
Hauts-de-France          53467.33
Auvergne-Rhône-Alpes     64856.25
Nouvelle-Aquitaine       71843.18
Île-de-France           127936.94
Name: ca, dtype: float64
```

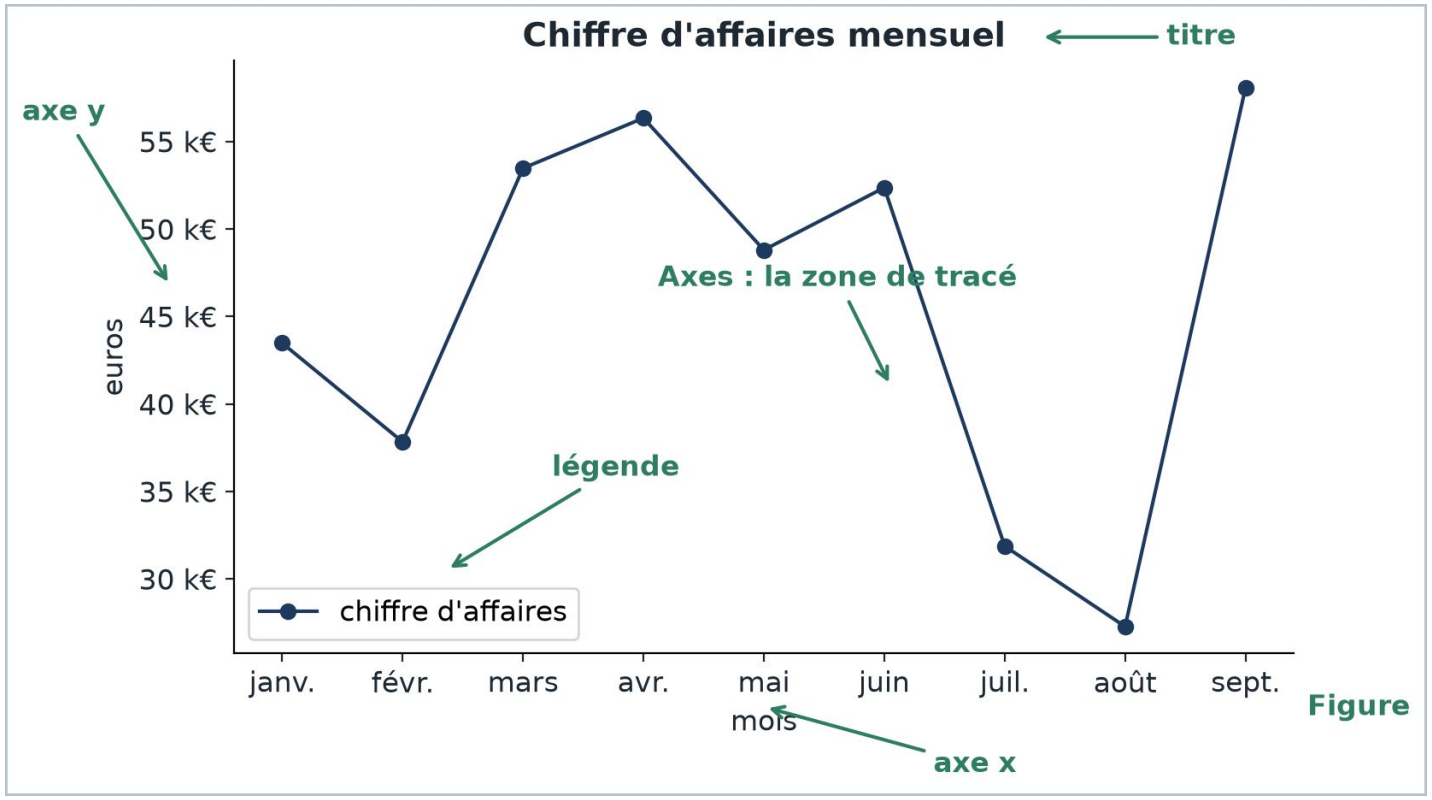
Dans tout le notebook : un **résultat attendu** par question, des cellules `# Vérification` qui affichent `OK`, et des questions **Pour aller plus loin** facultatives.

# 02

# matplotlib

Anatomie | Tracés de base | Repères | Grilles | Inspection

# 2 Anatomie d'une figure




| Élément | Rôle |
|---|---|
| `Figure` | l'image entière |
| `Axes` | une zone de tracé complète |
| `Axis` | un axe, x ou y |

Un `Axes` est un graphique, pas un axe. Deux graphiques côte à côte : une `Figure`, deux `Axes`.

# 2 Écrire avec des objets

```
# une figure de 8 pouces sur 4, un Axes
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(mensuel.index, mensuel.values)
ax.set_title("Chiffre d'affaires mensuel")
```

`plt.subplots()` renvoie la figure et son `Axes`
On agit ensuite sur `ax` : claire, cette écriture reste lisible avec plusieurs graphiques
L'écriture directe `plt.plot(...)` agit sur le « graphique courant » : réservée aux tracés rapides
Dans un script, terminer par `plt.show()`

# 2 Les tracés de base

| Méthode | Pour montrer |
|---|---|
| `ax.plot(x, y)` | une évolution |
| `ax.bar(x, h)` | une comparaison entre catégories |
| `ax.barh(y, l)` | idem, avec des libellés longs |
| `ax.hist(v, bins=30)` | une distribution |
| `ax.scatter(x, y)` | une relation |

```
fig, ax = plt.subplots(figsize=(8, 4))
ax.barh(par_region.index, par_region.values,
        color="#2F7D62")
ax.set_title("Chiffre d'affaires par région,
"
             "janvier à septembre 2025")
ax.set_xlabel("chiffre d'affaires (euros)")
```

Avec `barh`, la première valeur est tracée en bas : triée par ordre croissant, la Series place la plus grande barre en haut.

# 2 Régler un histogramme et un nuage de points

```
# 16 classes de 10 points, de 20 % à 180 %
ax.hist(indicateurs["taux_realisation"],
        bins=[x / 10 for x in range(2, 19)],
        edgecolor="white")

# des points semi-transparents
ax.scatter(indicateurs["nb_commandes"],
           indicateurs["ca"], alpha=0.7)
```

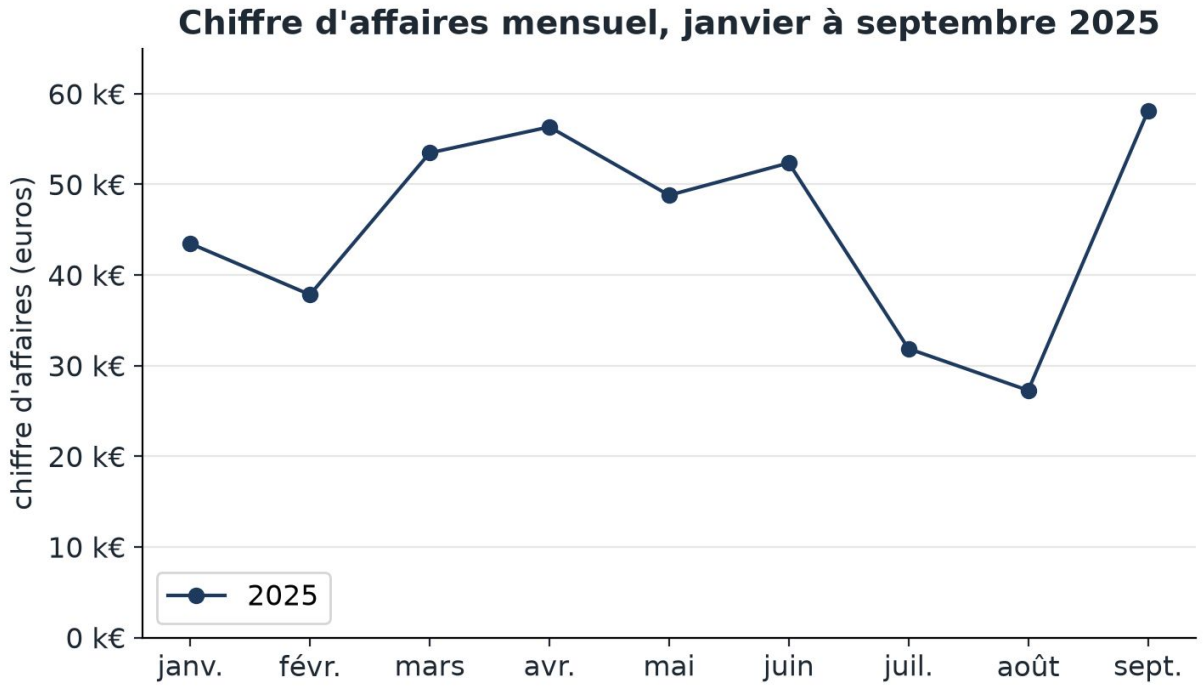
`bins` : un nombre de classes, ou la liste de leurs bornes pour des classes alignées sur des valeurs rondes
`edgecolor="white"` sépare les barres par un contour blanc
`alpha` : la transparence, de 0 (invisible) à 1 (opaque) ; les points qui se superposent restent visibles

La liste des bornes se construit avec une compréhension de liste, vue le premier jour.

# 2 Personnaliser

```
ax.plot(mensuel.index, mensuel.values,
        color="#1F3A5F", marker="o",
        label="2025")
ax.set_title("Chiffre d'affaires mensuel")
ax.set_xlabel("mois")
ax.set_ylabel("chiffre d'affaires (euros)")
ax.set_ylim(0, 65000)   # ax.set_xlim pour x
ax.legend()             # utilise les label=
ax.grid(axis="y", alpha=0.3)
ax.tick_params(axis="x", labelrotation=45)
```

`ax.legend(loc="upper left", ncols=2)` : en haut à gauche, sur deux colonnes. Si la légende cache des données, on agrandit l'axe avec `set_ylim`.



# 2 Des nombres lisibles : les formateurs

```
from matplotlib.ticker import FuncFormatter

milliers = FuncFormatter(
    lambda v, _: f"{v / 1000:.0f} k€"
)
ax.yaxis.set_major_formatter(milliers)

pourcentage = FuncFormatter(
    lambda v, _: f"{v:.0%}"
)
ax.xaxis.set_major_formatter(pourcentage)
```

| Graduation | Affichée |
|---|---|
| `45000` avec `milliers` | `45 k€` |
| `1.2` avec `pourcentage` | `120%` |

La fonction reçoit la valeur de la graduation et sa position, ignorée avec _

- Rangé dans une variable, le formateur se réutilise sur chaque graphique

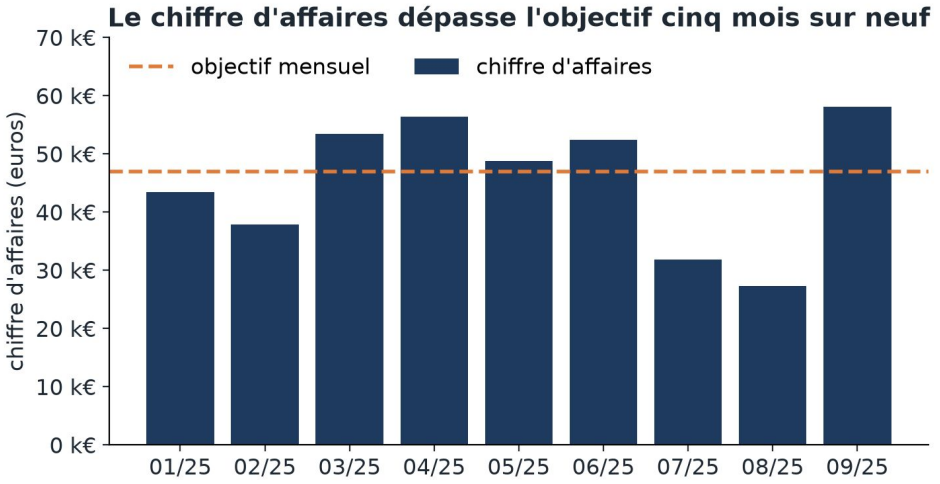
On retrouve la `lambda` et les formats des f-strings du premier matin

# 2 Lignes de référence

Un repère donne du sens à une valeur : un objectif, une moyenne, un seuil.

```
objectif_total = (
    indicateurs.groupby("mois")["objectif_mensuel"]
    .sum().iloc[0]
)   # 47 000 euros

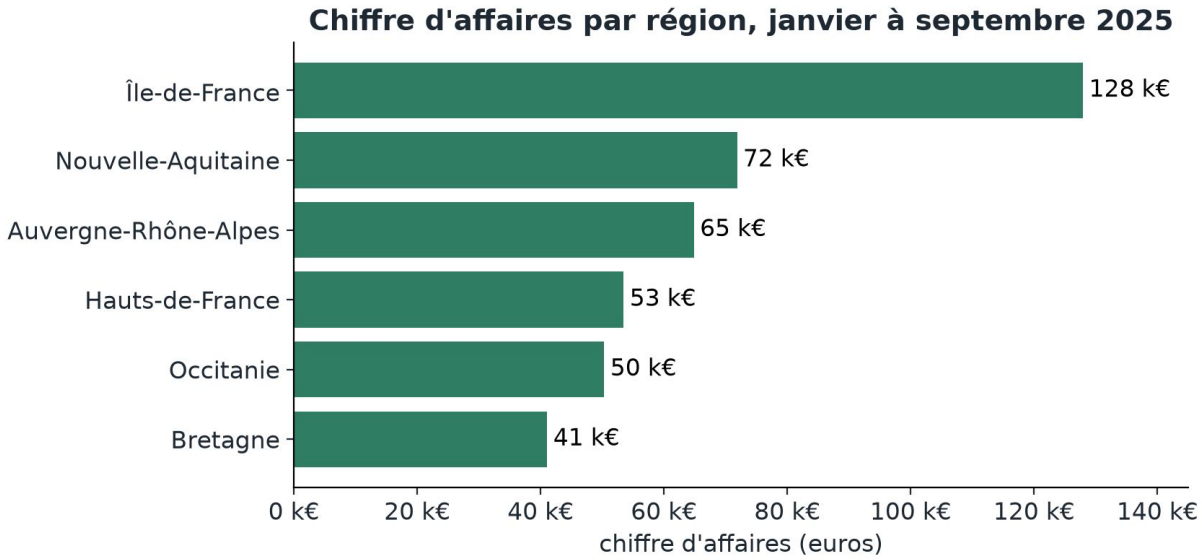
ax.axhline(objectif_total, color="#E07B39",
           linestyle="--", label="objectif mensuel")
ax.axvline(1, color="#E07B39", linestyle="--")
ax.axvspan(pd.to_datetime("2025-06-15"),
           pd.to_datetime("2025-08-15"),
           color="#B8C2CC", alpha=0.3)
```




`axhline` : une ligne horizontale sur toute la largeur
`axvline` : une ligne verticale, ici à x = 1
`axvspan` : une bande verticale entre deux abscisses, ici l'été sur un axe de dates

# 2 Écrire les valeurs avec bar_label

```
barres = ax.barh(par_region.index,
                 par_region.values)
ax.bar_label(
    barres,
    labels=[f"{v / 1000:.0f} k€"
            for v in par_region.values],
    padding=3,
)
ax.set_xlim(0, 145000)   # de la place
```



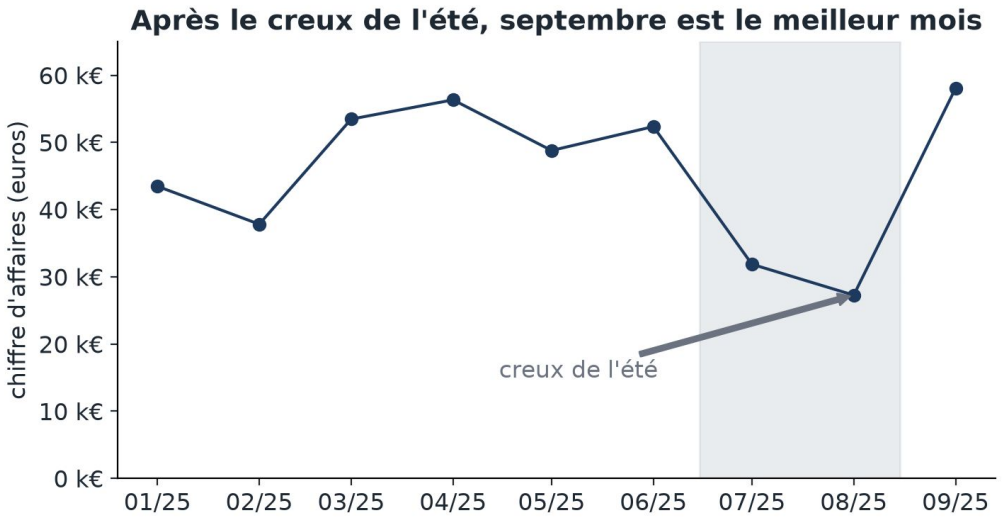

`ax.bar` et `ax.barh` renvoient les barres tracées
Sans `labels`, la valeur brute ; `padding` écarte le label de la barre
Un label dépasse du cadre ? On prolonge l'axe avec `set_xlim` ou `set_ylim`

# 2 Pointer un fait marquant avec annotate

```
ax.annotate(
    "creux de l'été",
    xy=(mensuel.index[7], mensuel.iloc[7]),
    xytext=(pd.to_datetime("2025-04-15"), 15000),
    arrowprops={"arrowstyle": "simple",
                "color": "#6B7280"},
    color="#6B7280",
)
```

`xy` : le point désigné, ici août
`xytext` : la position du texte, relié au point par une flèche

- Les coordonnées sont dans les unités des axes : des dates en abscisse, des euros en ordonnée




La bande grise vient de `axvspan`, et le titre énonce le message.

# 2 Plusieurs graphiques dans une figure

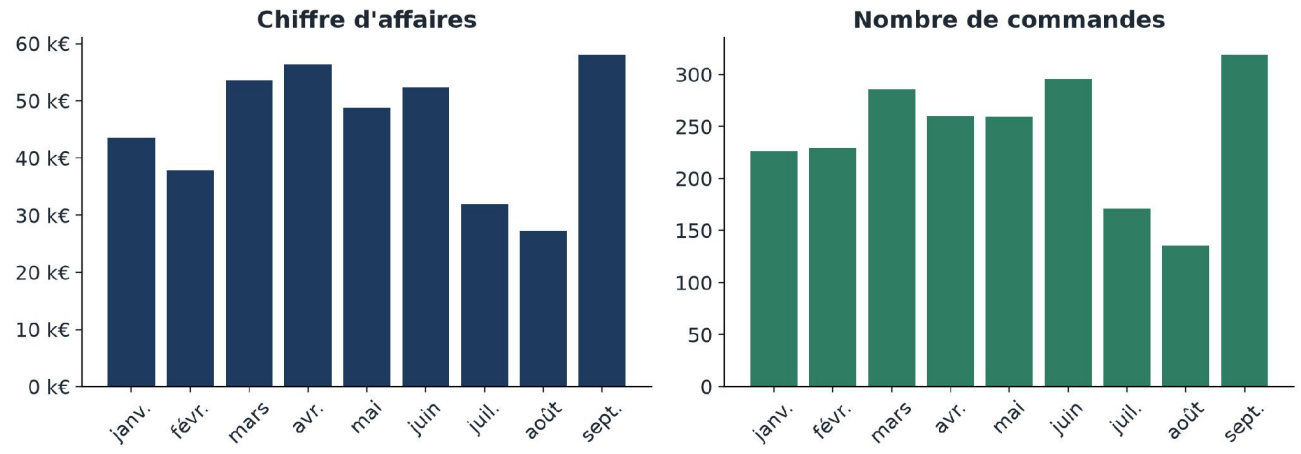
```
# commandes : nombre de commandes par mois
etiquettes = mensuel.index.strftime("%m/%y")

fig, axes = plt.subplots(1, 2,
                         figsize=(11, 4))

axes[0].bar(etiquettes, mensuel.values)
axes[0].set_title("Chiffre d'affaires")

axes[1].bar(etiquettes, commandes.values)
axes[1].set_title("Nombre de commandes")

fig.tight_layout()
```



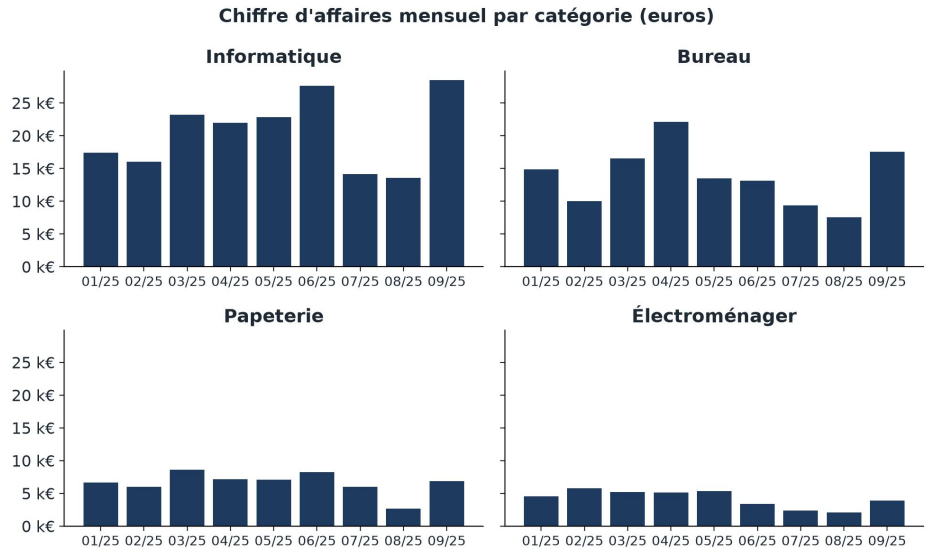

`plt.subplots(2, 3)` : une grille, `axes[1, 2]` pour une case
`tight_layout` évite les chevauchements
`sharey=True` : même échelle verticale, indispensable pour comparer ;
`sharex=True` pour l'axe horizontal

# 2 Remplir une grille : axes.flat

`axes.flat` énumère les graphiques ligne par ligne, `zip` les associe aux données

```
ca_categories = ventes.pivot_table(
    index=ventes["date"].dt.strftime("%m/%y"),
    columns="categorie", values="montant",
    aggfunc="sum",
)
categories = ["Informatique", "Bureau",
              "Papeterie", "Électroménager"]

fig, axes = plt.subplots(2, 2, figsize=(10, 6),
                         sharey=True)
for ax, categorie in zip(axes.flat, categories):
    ax.bar(ca_categories.index,
           ca_categories[categorie])
    ax.set_title(categorie)
fig.suptitle("Chiffre d'affaires mensuel par catégorie")
fig.tight_layout()
```




Avec `sharey=True`, l'informatique pèse environ cinq fois plus que l'électroménager ; sans, on ne le verrait pas.

# 2 Enregistrer, et le raccourci pandas

## Enregistrer

```
fig.savefig("ca_mensuel.png", dpi=150,
            bbox_inches="tight")
fig.savefig("ca_mensuel.svg")
```

`dpi` : 150 pour un écran, 300 pour une impression. Le SVG reste net à toutes les tailles.

## pandas trace aussi

```
mensuel.plot(title="CA mensuel")
par_region.plot.barh()

fig, ax = plt.subplots()
mensuel.plot(ax=ax)
```

pandas place l'index sur l'axe des catégories et le nomme avec le nom de l'index. Pratique pour explorer ; pour un graphique de restitution, on reprend la main avec `ax`.

# 2 Inspecter un graphique par le code

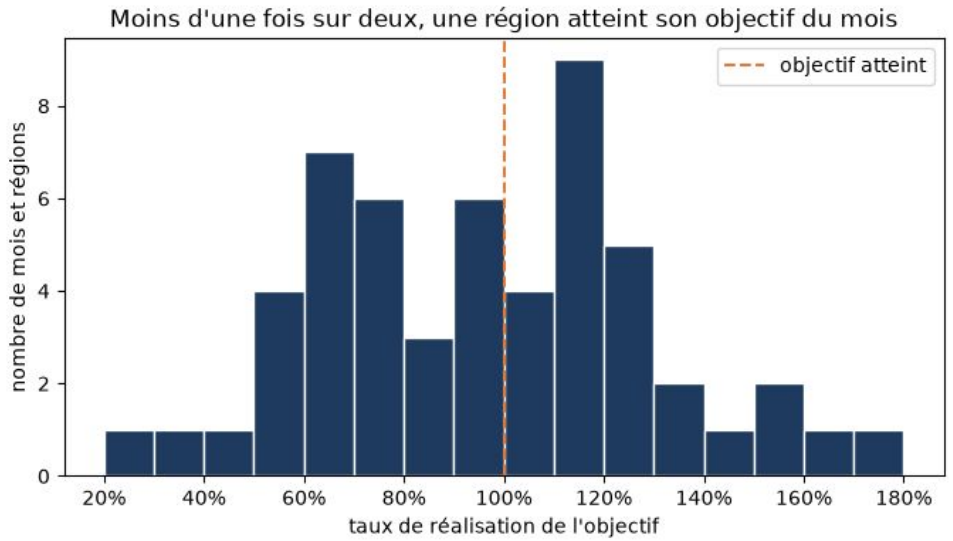
Un graphique matplotlib est un objet : on vérifie son contenu sans le regarder.

```
ax.get_title()                      # le titre
ax.get_xlabel(), ax.get_ylabel()    # les noms des axes
ax.get_ylim()                       # (bas, haut) de l'axe vertical
ax.patches                          # les barres : get_height(), get_width(), get_facecolor()
ax.lines                            # les courbes et les lignes de référence
ax.texts                            # les textes ajoutés : labels de bar_label, annotations
ax.collections                      # les points d'un nuage, les cases d'une heatmap
fig.axes                            # la liste des Axes de la figure
```

Les vérifications du TP4 s'en servent : `assert ax.get_ylim()[0] == 0` pour un axe qui part de zéro, `len(ax.patches) == 9` pour une barre par mois. D'où la consigne : toujours nommer la figure `fig` et le graphique `ax`.

# 2 Pratique : TP4, partie 1

| Question | À tracer |
|---|---|
| 1.1 | la courbe du chiffre d'affaires mensuel, avec le formateur `milliers` |
| 1.2 | les barres mensuelles et la ligne d'objectif |
| 1.3 | les régions en barres horizontales, avec `bar_label` |
| 1.4 | chiffre d'affaires et commandes côte à côte, enregistrés en PNG |
| 1.5 | l'histogramme du taux de réalisation |
| 1.6 | le nuage de points commandes et chiffre d'affaires |
| 1.7 | une grille de quatre graphiques, `sharey=True` |
| 1.8, 1.9 | facultatives : `axvspan` et `annotate`, le raccourci pandas |




Résultat attendu de la question 1.5

Garder les noms `fig` et `ax` : les vérifications les inspectent.

# 03

# seaborn

Distributions | Catégories | Corrélations | Petits multiples

# 3 Le principe de seaborn

```
sns.set_theme(style="whitegrid")

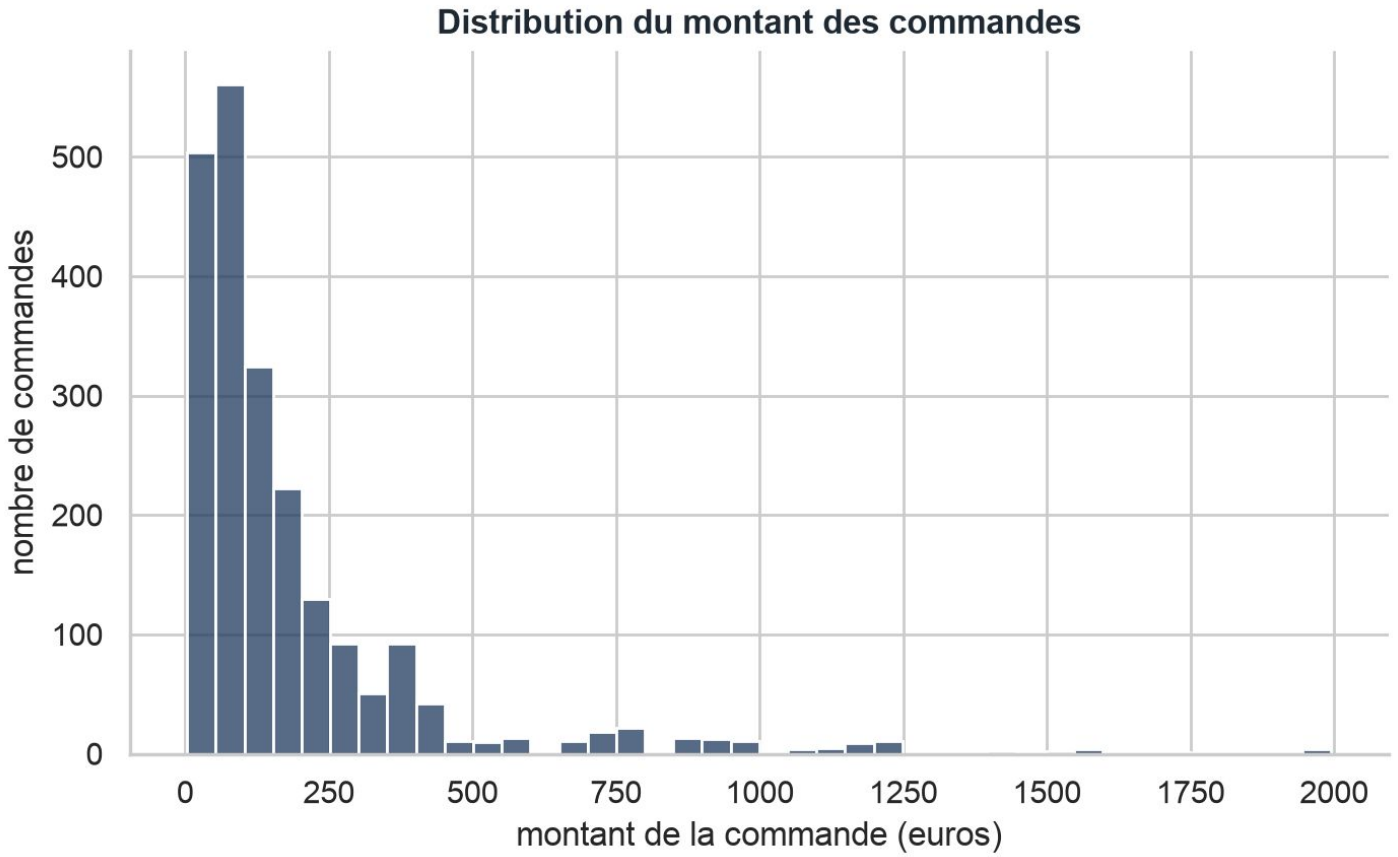
fig, ax = plt.subplots(figsize=(9, 4.5))
sns.lineplot(data=indicateurs, x="mois", y="ca",
             hue="region", marker="o", ax=ax)
# la légende à droite, hors du graphique
sns.move_legend(ax, "upper left",
                bbox_to_anchor=(1, 1))
```

Toujours passer `ax=ax` : sinon seaborn dessine dans le dernier graphique ouvert.

| Paramètre | Rôle |
|---|---|
| `data` | le DataFrame |
| `x`, `y` | les colonnes des axes |
| `hue` | la colonne de la couleur |
| `color` | une couleur unique, sans `hue` |
| `ax` | l'`Axes` où dessiner |

Chaque fonction renvoie un `Axes` matplotlib : on le personnalise comme vu plus haut. `set_theme` change le style de tous les graphiques qui suivent, matplotlib compris.

# 3 La distribution des montants




```
sns.histplot(data=ventes,
             x="montant",
             bins=40, ax=ax)
```

**188 €**

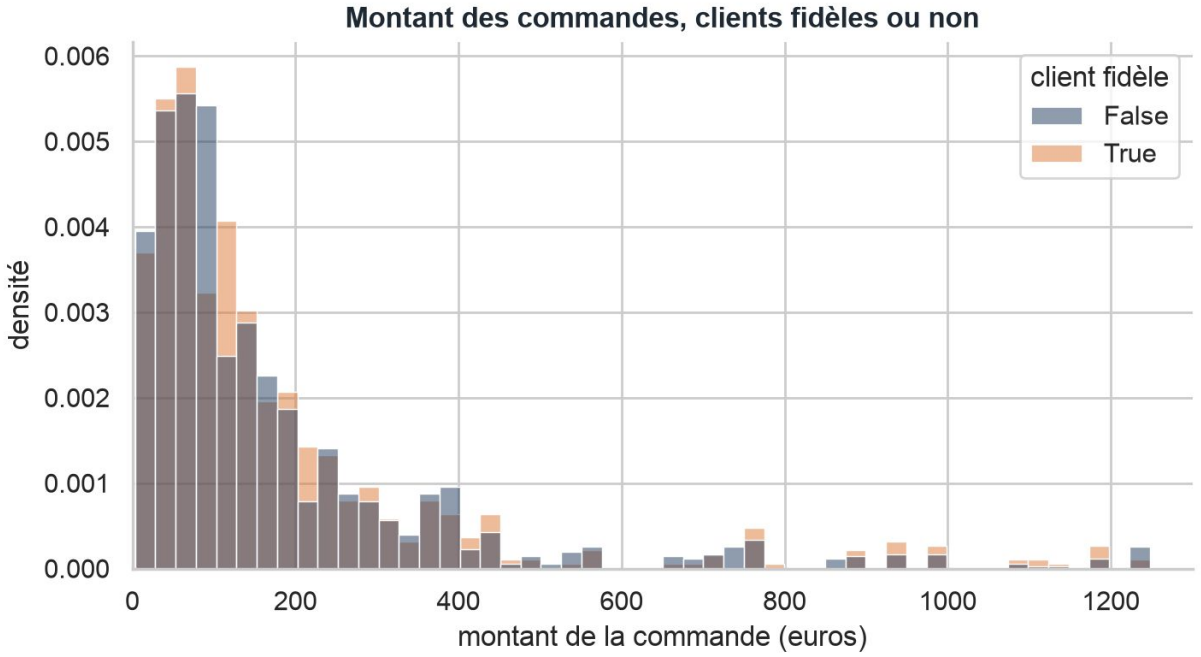
moyenne

**105 €**

médiane

Beaucoup de petites commandes, quelques grosses : la moyenne est tirée vers le haut.

# 3 Comparer deux distributions : des densités



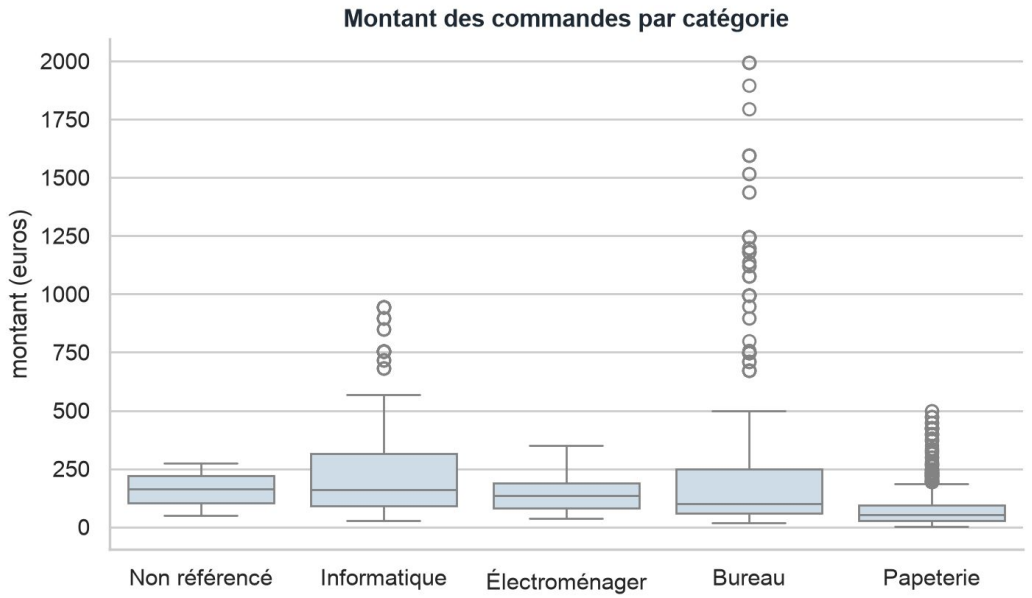

```
sns.histplot(data=ventes, x="montant",
             hue="client_fidele", ax=ax)

# des densités, normalisées par groupe
sns.histplot(data=ventes, x="montant",
             hue="client_fidele",
             stat="density",
             common_norm=False, ax=ax)
```

- Près de deux fois plus de commandes de clients non fidèles : les effectifs ne se comparent pas

`stat="density"` et `common_norm=False` : des densités normalisées séparément, pour comparer les formes

# 3 Lire une boîte à moustaches




```
ordre = (ventes.groupby("categorie")["montant"]
         .median().sort_values(ascending=False)
         .index)
sns.boxplot(data=ventes, x="categorie",
            y="montant", order=ordre, ax=ax)
```

`order` fixe l'ordre des catégories, ici par médiane décroissante.

**Boîte** : la moitié centrale des valeurs
**Trait** : la médiane
**Moustaches** : jusqu'à 1,5 fois la hauteur de la boîte
**Points** : les valeurs extrêmes

Bureau : les valeurs extrêmes les plus élevées. Papeterie : beaucoup de points isolés, car sa boîte est si étroite que 200 € dépasse déjà ses moustaches.

# 3 Des barres par catégorie

```
sns.barplot(data=ventes, x="region", y="montant",
            estimator="sum", errorbar=None, ax=ax)

sns.countplot(data=ventes, x="categorie",
              order=ventes["categorie"]
                    .value_counts().index,
              ax=ax)
```

Par défaut, `barplot` affiche la **moyenne** de chaque groupe, avec un intervalle de confiance. Pour un total : `estimator="sum"`, et `errorbar=None` pour retirer le trait.

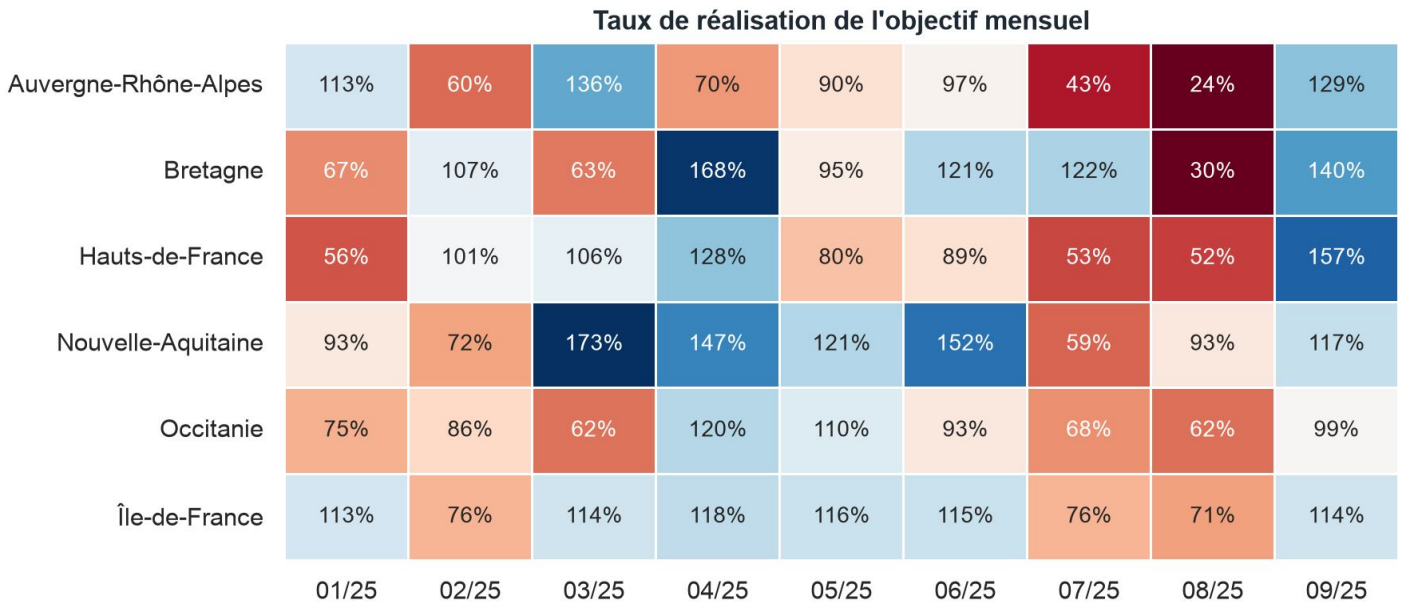
Quand les données sont déjà agrégées, comme la table des indicateurs, matplotlib ou pandas suffisent.

`countplot` compte les lignes de chaque catégorie `order` avec l'index de `value_counts` : de la plus haute à la plus basse

La papeterie est la deuxième catégorie en nombre de commandes, mais seulement la troisième en chiffre d'affaires.

# 3 La heatmap

```
taux = indicateurs.pivot_table(
    index="region",
    columns=indicateurs["mois"]
        .dt.strftime("%m/%y"),
    values="taux_realisation",
)
sns.heatmap(taux, annot=True,
            fmt=".0%", cmap="RdBu",
            center=1, cbar=False,
            ax=ax)
```




`center=1` place la couleur neutre sur 100 % : en rouge sous l'objectif, en bleu au-dessus

`cbar=False` retire la barre de couleur, inutile quand les valeurs sont écrites

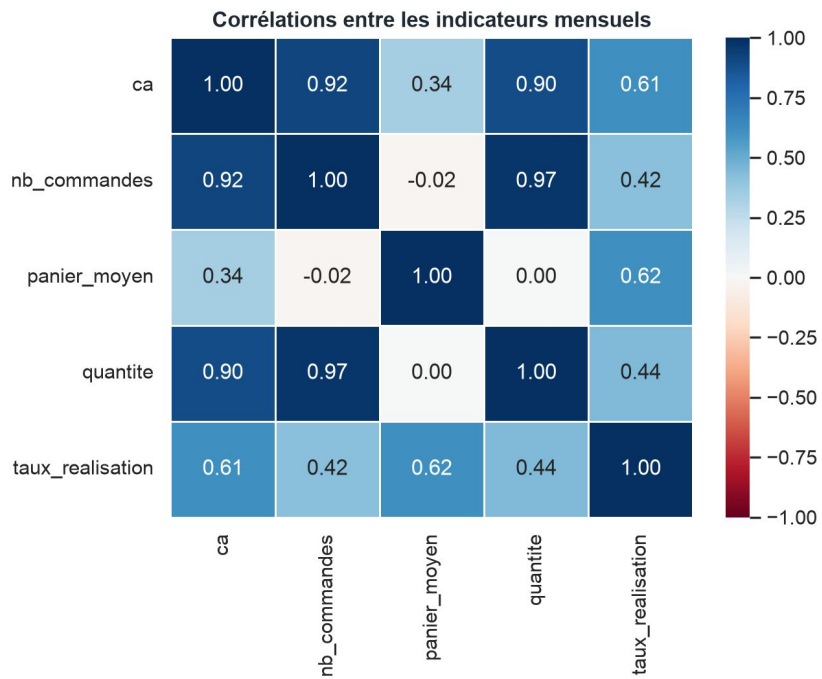
# 3 La matrice de corrélation

```
colonnes = ["ca", "nb_commandes", "panier_moyen",
            "quantite", "taux_realisation"]
correlations = indicateurs[colonnes].corr()

sns.heatmap(correlations, annot=True, fmt=".2f",
            cmap="RdBu", vmin=-1, vmax=1, ax=ax)
```

- Le coefficient mesure à quel point deux colonnes varient ensemble, de façon linéaire

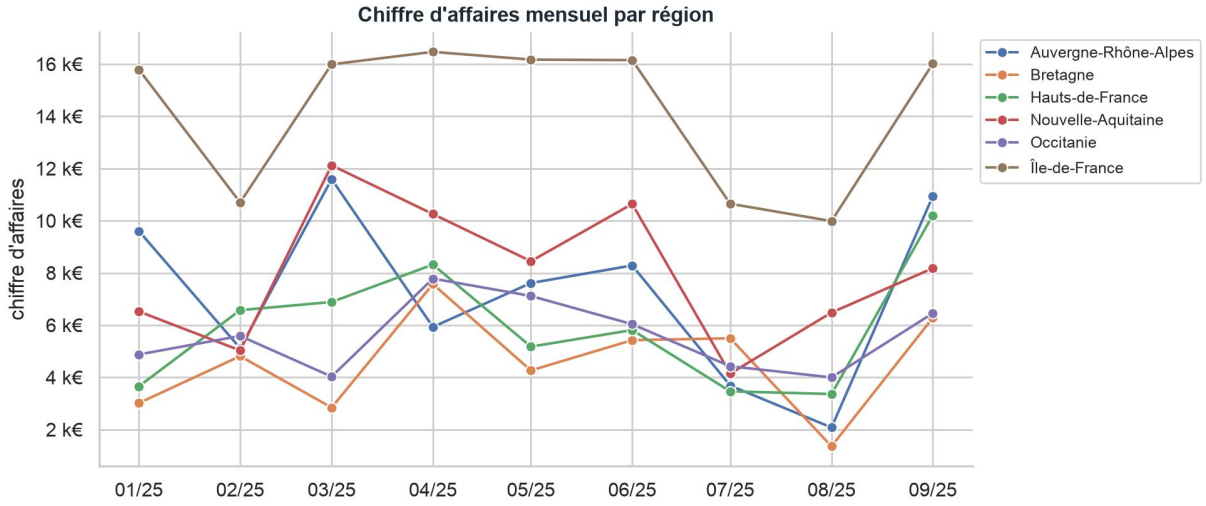
**1** : elles augmentent ensemble ; **-1** : l'une baisse quand l'autre monte ;
**0** : pas de lien linéaire
`vmin` et `vmax` : la couleur garde le même sens d'un tableau à l'autre




Chiffre d'affaires et commandes : 0,92. Panier moyen et commandes : -0,02.

Une corrélation mesure une liaison, pas une cause.

# 3 Un graphique par groupe




Six courbes superposées, un « plat de spaghettis » : on ne suit aucune région.

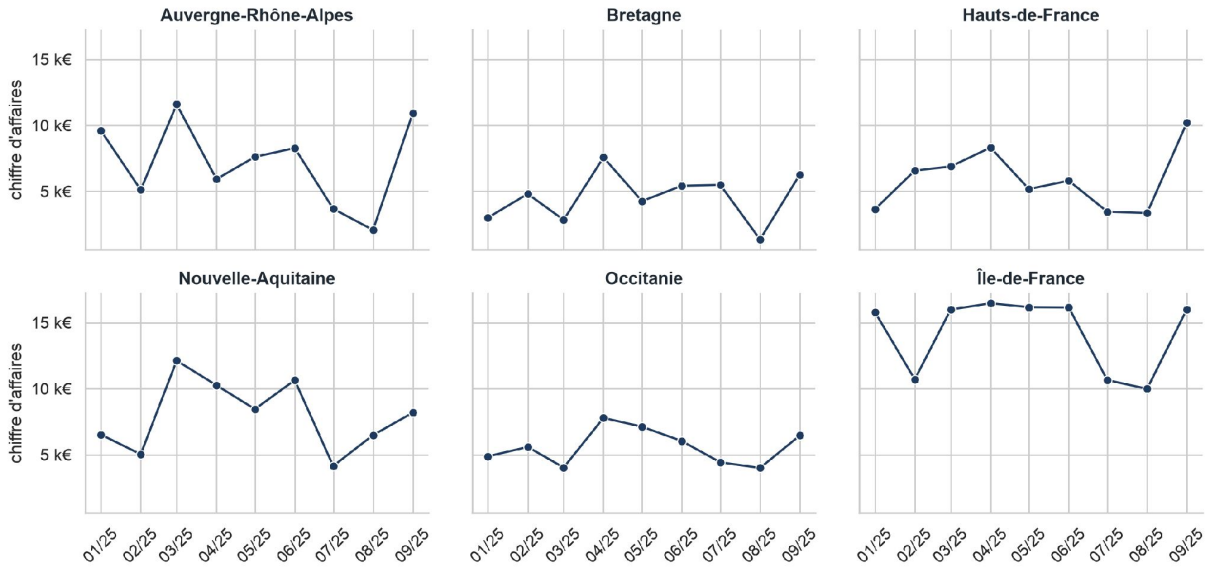
```
sns.relplot(data=indicateurs,
            x="mois", y="ca",
            col="region", col_wrap=3,
            kind="line")
```

`relplot`, `displot`, `catplot` créent leur propre figure
`col` : un graphique par valeur, avec la même échelle
On parle de **petits multiples**

# 3 Régler la grille de relplot

```
g = sns.relplot(
    data=indicateurs, x="mois", y="ca",
    col="region", col_wrap=3, kind="line",
    height=2.8, aspect=1.4,
)
g.set_titles("{col_name}")
g.set_axis_labels("", "chiffre d'affaires")
g.tick_params(axis="x", labelrotation=45)
for ax in g.axes.flat:
    ax.yaxis.set_major_formatter(milliers)
```

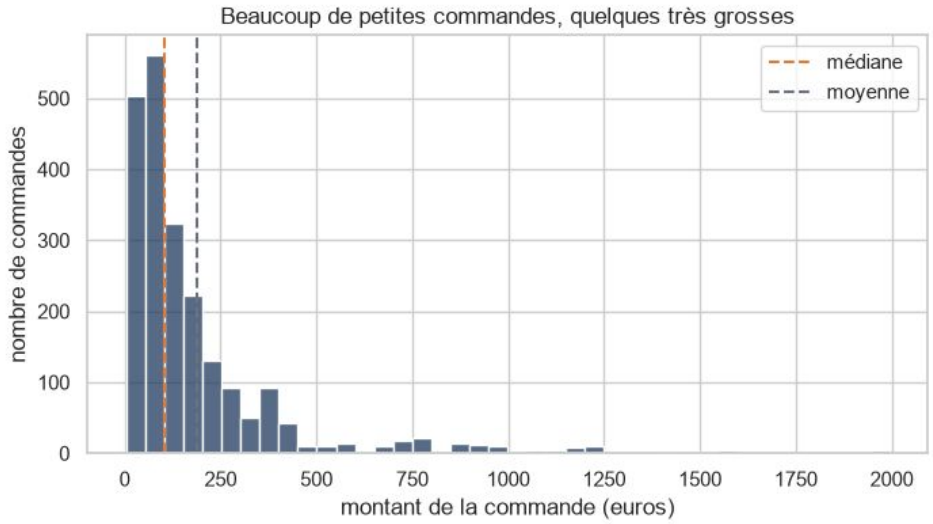
`relplot` renvoie une grille (`FacetGrid`), pas un `Axes` : pas de `ax=`
`height` : hauteur d'un graphique en pouces ; `aspect` : largeur sur hauteur
`g.axes.flat` parcourt les graphiques, comme `axes.flat`




Même échelle verticale par défaut : l'Île-de-France se détache nettement des autres régions.

# 3 Pratique : TP4, partie 2

| Question | À tracer |
|---|---|
| 2.1 | l'histogramme des montants, avec moyenne et médiane |
| 2.2 | les boîtes à moustaches, triées par médiane |
| 2.3, 2.4 | commandes et chiffre d'affaires par catégorie |
| 2.5 | six courbes superposées, avec `move_legend` |
| 2.6 | les petits multiples avec `relplot` |
| 2.7 | la heatmap du taux de réalisation |
| 2.8 à 2.10 | facultatives : corrélations, densités, `barplot` par défaut |




Résultat attendu de la question 2.1

La première cellule de la partie applique `sns.set_theme(style="whitegrid")` à tous les graphiques.

# 04

# plotly

Interactivité | Usages courants | Info-bulles | Export HTML

# 4 Des graphiques interactifs

```
fig = px.line(
    indicateurs,
    x="mois",
    y="ca",
    color="region",
    markers=True,
    title="Chiffre d'affaires mensuel par région",
)
fig.show()
```

| Fonction | Graphique |
|---|---|
| `px.line` | courbes |
| `px.bar` | barres |
| `px.scatter` | nuage |
| `px.histogram` | histogramme, comptage |
| `px.box` | boîte |

Survol, zoom, légende cliquable pour masquer une série.

# 4 Quelques usages courants

```
px.bar(indicateurs, x="mois", y="ca",
       color="region")       # barres empilées
px.scatter(indicateurs, x="nb_commandes", y="ca",
           color="region", size="quantite")
px.box(ventes, x="categorie", y="montant",
       hover_data=["produit", "quantite"])

jours = ["lundi", "mardi", "mercredi", "jeudi",
         "vendredi", "samedi", "dimanche"]
px.histogram(ventes, x="jour",
             category_orders={"jour": jours})
```

`color` dans `px.bar` : des barres empilées, le total du mois et sa décomposition
`size` : la taille des points porte une troisième grandeur
`hover_data` dans `px.box` : on identifie une commande au survol, sans filtre
`px.histogram` sur une colonne texte compte les lignes ;
`category_orders` impose l'ordre du calendrier

Un graphique plotly est un objet : `fig.data` contient les traces, une par couleur, et `fig.layout` la mise en page.
Les vérifications du TP4 testent par exemple `len(fig.data) == 6` ou `fig.layout.title.text`.

# 4 Des info-bulles utiles

```
fig = px.line(
    indicateurs, x="mois", y="ca", color="region",
    hover_data={"nb_commandes": True,
                "taux_realisation": ":.0%"},
    labels={"ca": "chiffre d'affaires",
            "mois": "", "region": "région"},
)
```

`hover_data` ajoute des colonnes à l'info-bulle, avec un format
`labels` remplace les noms de colonnes partout dans le graphique

- L'information secondaire va dans l'info-bulle : le graphique reste léger

Pour une date, le format commence par une barre verticale : `"|%m/%Y"` affiche `03/2025`

# 4 Personnaliser et exporter en HTML

```
fig.update_layout(
    yaxis_title="chiffre d'affaires (euros)",
    legend_title="",
    template="simple_white",
)
fig.update_traces(line_width=2)

fig.write_html("ca_mensuel.html")
fig.write_html("ca_mensuel_cdn.html",
               include_plotlyjs="cdn")
Path("ca_mensuel_cdn.html").stat().st_size
```

**4,8 Mo**

par défaut, autonome

**14 Ko**

avec `"cdn"`

Le fichier s'ouvre dans n'importe quel navigateur, sans Python, et reste interactif. Avec `"cdn"`, il ne contient plus que le graphique, mais a besoin d'internet pour charger plotly.

Le modèle se passe aussi à la création : `px.bar(..., template="simple_white")`.

# 4 Statique ou interactif ?

| Usage | Choix |
|---|---|
| rapport imprimé, PDF, diapositive | statique : matplotlib, seaborn |
| exploration, présentation à l'écran | interactif : plotly |
| partage d'un fichier à ouvrir soi-même | interactif : export HTML |
| beaucoup de séries ou de points | interactif, pour zoomer et filtrer |

# 4 Pratique : TP4, partie 3

Les graphiques plotly sont interactifs : survoler, zoomer, cliquer sur la légende.

| Question | Variable | À construire |
|---|---|---|
| 3.1 | `fig_ca` | `px.line` par région, info-bulle enrichie avec `hover_data`, libellés avec `labels` |
| 3.2 | `fig_barres` | `px.bar`, des barres empilées par région |
| 3.3 | `fig_boites` | `px.box` par catégorie, produit et quantité au survol |
| 3.4 | `fig_nuage` | `px.scatter` avec `size="quantite"`, le mois au format `|%m/%Y` |
| 3.5 | | deux exports HTML de `fig_ca` et leur taille en kilo-octets |
| 3.6 | `fig_jours` | facultative : les commandes par jour, avec `category_orders` |

Les vérifications inspectent la figure : `len(fig_ca.data) == 6` pour une courbe par région, `fig_ca.layout.title.text` pour le titre.

# 05

# Bonnes pratiques de dataviz

Choisir | Rendre lisible | Ne pas tromper | Daltonisme

# 5 Partir de la question

| Question | Graphique |
|---|---|
| Comment évolue une valeur dans le temps ? | courbe |
| Quelle catégorie est la plus grande ? | barres triées |
| Comment se répartissent les valeurs ? | histogramme, boîte à moustaches |
| Deux grandeurs sont-elles liées ? | nuage de points |
| Comment un total se décompose-t-il ? | barres empilées ; un camembert seulement pour deux ou trois parts |
| Comment un indicateur varie-t-il selon deux dimensions ? | heatmap |

# 5 Rendre le graphique lisible

### 1 Un titre qui dit le message

« L'Île-de-France réalise près d'un tiers du chiffre d'affaires » plutôt que « CA par région ».

### 2 Des axes et des nombres clairs

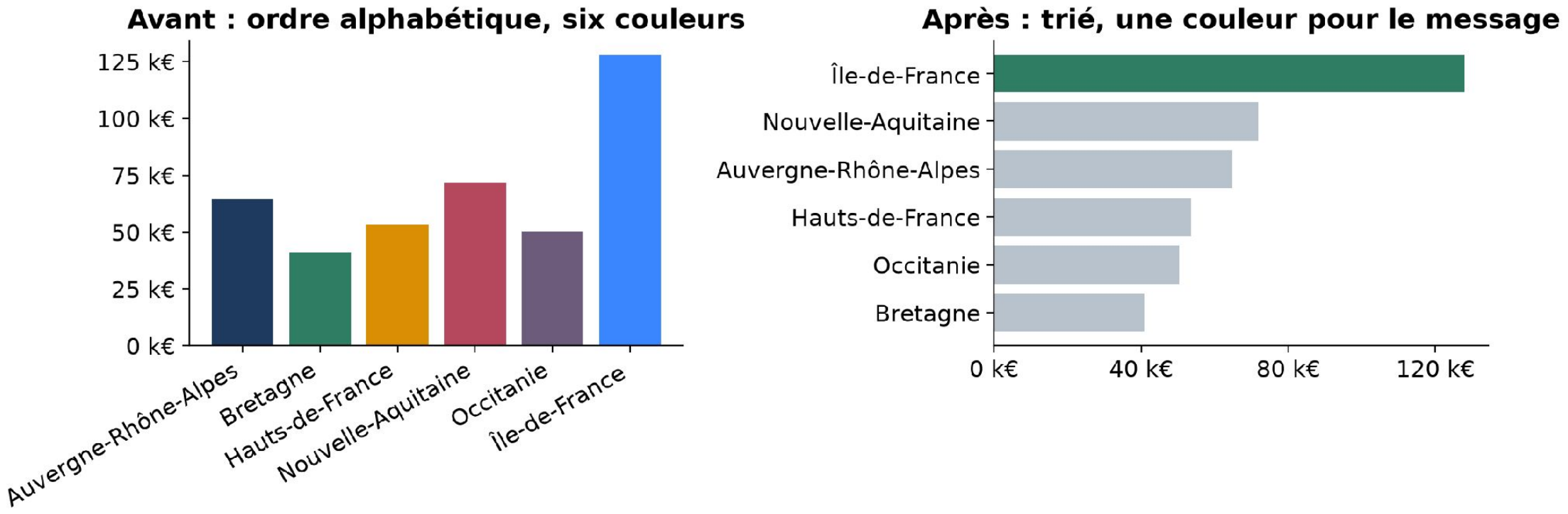
Chaque axe nommé avec son unité, et 50 k€ plutôt que 50000.

### 3 Un ordre qui aide

Des barres triées, horizontales si les libellés sont longs, sauf quand l'ordre a un sens : mois, jours de la semaine, tranches d'âge.

Peu de couleurs : une couleur n'a de sens que si elle porte une information.

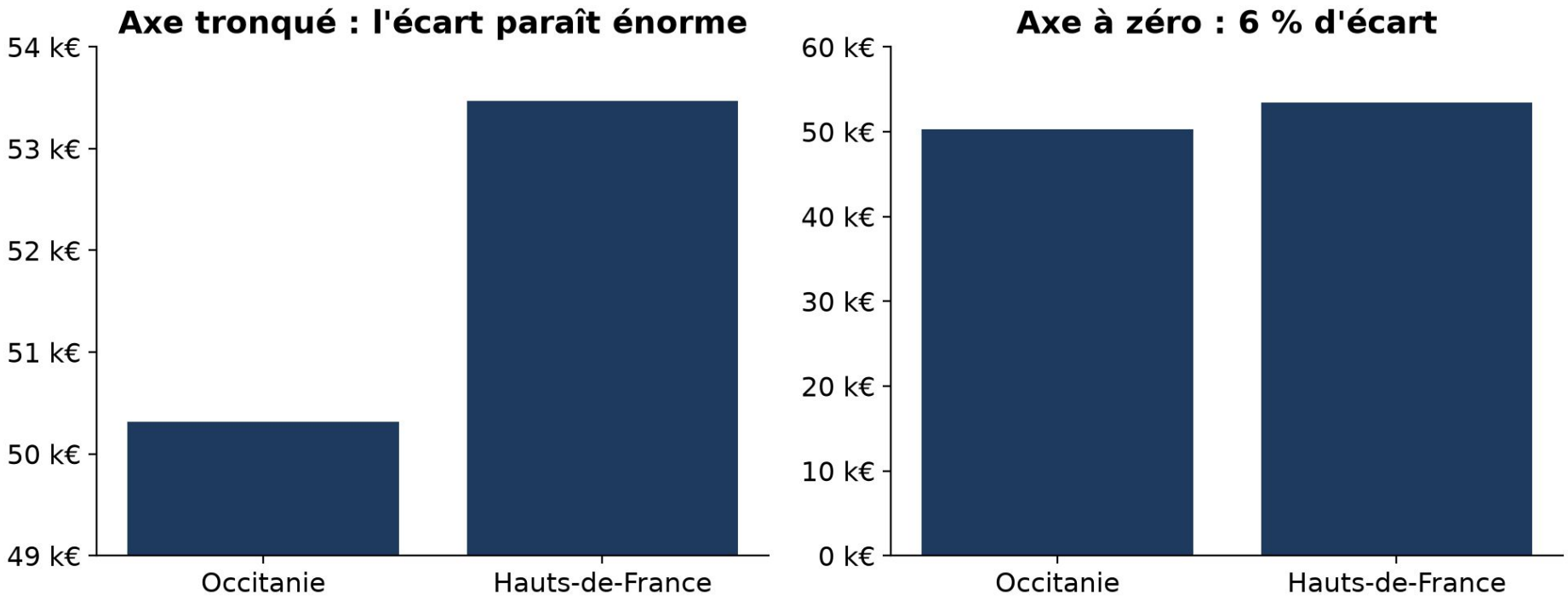
# 5 Une couleur pour le message, du gris pour le reste




```
couleurs = ["#2F7D62" if r == "Île-de-France" else "#B8C2CC" for r in par_region.index]
ax.barh(par_region.index, par_region.values, color=couleurs)
```

5

# Le piège de l'axe tronqué




La longueur d'une barre représente la valeur : un axe en barres part toujours de zéro. Pour une courbe, un axe qui ne part pas de zéro est acceptable s'il est visible.

# 5 D'autres graphiques trompeurs

1

### Un ordre arbitraire

L'ordre alphabétique oblige à chercher les plus grandes valeurs.

2

### Des couleurs décoratives

Une couleur par barre, et une légende qui répète l'axe.

3

### Des échelles différentes

Deux graphiques comparés sans `sharey=True`.

4

### Deux axes verticaux

Le choix des échelles suggère des liens qui n'existent pas.

5

### La 3D et les camemberts

L'œil compare mal les angles et les volumes.

6

### Une période incomplète

Un dernier mois partiel fait croire à une chute : l'exclure ou le signaler.

# 5 Reconnaître un graphique trompeur dans le code

```
ax.set_ylim(150, 235)                              # sur des barres : axe tronqué
ax2 = ax.twinx()                                   # un second axe vertical, qui partage l'axe x
ax.pie(valeurs, labels=noms, autopct="%.0f%%")     # camembert, avec le pourcentage de chaque part
```

`twinx` crée un deuxième `Axes` superposé au premier, avec sa propre échelle verticale.

| Plutôt que | On trace |
|---|---|
| deux axes verticaux avec `twinx` | deux graphiques l'un au-dessus de l'autre, alignés sur le même axe horizontal : `plt.subplots(2, 1, sharex=True)` |
| un camembert à nombreuses parts | des barres horizontales triées, avec la part de chacune écrite par `bar_label` |
| un axe tronqué sur des barres | un axe qui part de zéro : `ax.set_ylim(0, ...)` |

# 5 Penser aux daltoniens

**8 %**

des hommes ont une forme de daltonisme

Le plus souvent : confusion entre le **rouge** et le **vert**
Éviter de les opposer, par exemple pour « sous » et « au-dessus de l'objectif », et la palette `"RdYlGn"` (rouge, jaune, vert)

- Palettes sûres :

`"RdBu"` pour une échelle autour d'une valeur centrale
`"viridis"`, `"cividis"` pour une échelle continue
`"colorblind"` dans seaborn pour des catégories

```
sns.set_theme(palette="colorblind")
```

# 5 Pratique : TP4, partie 4

Pour chaque graphique : l'exécuter tel quel, expliquer son défaut, puis le corriger

| Question | Le défaut | La correction attendue |
|---|---|---|
| 4.1 | le panier moyen par région, sur un axe tronqué | un axe qui part de zéro |
| 4.2 | les produits dans l'ordre alphabétique | des barres triées, le plus commandé en haut |
| 4.3 | une couleur par catégorie et une légende qui répète l'axe | une couleur pour la plus forte, du gris, pas de légende |
| 4.4 | deux régions sur des échelles différentes | `sharey=True`, une échelle qui part de zéro |
| 4.5 | un camembert à 17 parts | des barres triées et la part de chacune avec `bar_label` |
| 4.6, 4.7 | facultatives : deux axes avec `twinx`, la palette `"RdYlGn"` | `plt.subplots(2, 1, sharex=True)`, la palette `"RdBu"` |

06

# Construire un tableau de bord

Question, graphique, message | Ordre | Cohérence | Vérification

# 6 Une question, un graphique, un message

Un tableau de bord répond, en quelques images, aux questions de la personne qui le lit.

1

### La question

Celle du lecteur : « les régions atteignent-elles leurs objectifs ? »

2

### Le graphique

Son titre énonce la réponse.

3

### Ce qu'on retient

Une ou deux phrases, avec les chiffres clés.

Si on ne sait pas écrire la troisième partie, le graphique n'a pas sa place dans le tableau de bord.

# 6 Ordonner les graphiques et rester cohérent

## Du général au détail

1. **Évolution** — l'activité globale
2. **Répartition** — par région, par catégorie
3. **Objectifs** — leur suivi
4. **Détail** — exploré par le lecteur dans un graphique interactif

## Rester cohérent

- Une même chose garde la même couleur : l'Île-de-France en vert dans un graphique ne passe pas en orange dans le suivant
- Les mêmes unités et les mêmes formats partout : des k€ sur les montants, des pourcentages sans décimale
- Des échelles partagées dès que des graphiques se comparent

# 6 La liste de vérification

Avant de livrer, on relit chaque graphique avec cette liste :

- le type de graphique correspond à la question posée
- le titre énonce le message
- les axes sont nommés, avec leur unité, et les nombres sont lisibles
- les barres sont triées et partent de zéro
- la couleur sert à mettre en évidence, pas à décorer
- la palette est lisible par les daltoniens

La même liste figure en tête de la partie 5 du notebook.

# 6 Pratique : TP4, partie 5

4 à 5 graphiques qui racontent l'activité de janvier à septembre 2025 à la direction

| Graphique | La question de la direction |
|---|---|
| 1 | Comment le chiffre d'affaires a-t-il évolué depuis janvier ? |
| 2 | Quelles régions portent l'activité ? |
| 3 | Les régions atteignent-elles leurs objectifs ? |
| 4 | Comment chaque région a-t-elle évolué mois après mois ? Le graphique interactif principal, exporté en `ca_mensuel_regions.html` |
| 5, facultatif | Au choix, par exemple : quelles catégories font le chiffre d'affaires ? |

Pour chaque graphique : la question, le graphique, ce qu'on retient. Puis la liste de vérification.

# TP4 : tableau de bord d'indicateurs

Dans exercices/tp04-tableau-de-bord/tp04.ipynb, parties 0 à 5, au fil de l'après-midi

1

### S'exercer

Parties 0 à 4 : des graphiques guidés avec matplotlib, seaborn et plotly, puis des graphiques trompeurs à corriger.

2

### Composer

Partie 5 : 4 à 5 graphiques commentés, dont au moins un avec seaborn et un interactif avec plotly.

3

### Livrer

Le notebook de restitution, l'export HTML du graphique interactif principal et la figure `activite_mensuelle.png`.

Pour chaque graphique, un **résultat attendu** à reproduire ; des cellules de **vérification** contrôlent les réponses avec `assert`. Les questions **Pour aller plus loin** sont facultatives.

# À retenir

Une `Figure`, un ou plusieurs `Axes` : `fig, ax = plt.subplots()`
Titre, axes nommés avec unité, nombres lisibles avec `FuncFormatter`
`axhline`, `axvline` pour un repère, `bar_label`, `annotate`
`plt.subplots(2, 2, sharey=True)`, `axes.flat`, `tight_layout`
Un graphique s'inspecte : `get_title`, `get_ylim`, `patches`, `lines`
seaborn : `data`, `x`, `y`, `hue`, et toujours `ax=ax`
`order` pour trier ; `barplot` affiche une moyenne par défaut

`heatmap` sur un `pivot_table` ou une matrice de corrélation
`relplot` et `col` : des petits multiples
plotly : `hover_data`, `labels`, `size`, `write_html`

- Partir de la question, un titre qui énonce le message
- Une couleur pour le message, des barres triées qui partent de zéro, des échelles partagées
- Ni double axe, ni camembert à nombreuses parts, ni rouge contre vert
- Un tableau de bord : une question, un graphique, un message, du général au détail